Para resolver o aviso **"resource leak: 'scanner' is never closed"** (vazamento de recursos: o scanner nunca é fechado) no Java, você pode usar o comando `scanner.close()` ou a estrutura moderna **try-with-resources**. [1]

1. Usar o `scanner.close()`

Basta chamar o método de fechamento logo após terminar de usar o objeto:

java

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite algo:");
        String texto = scanner.nextLine();

        // Fecha o scanner e libera o recurso
        scanner.close();
    }
}
```

Use o código com cuidado.

2. Usar `try-with-resources` (Recomendado)

Disponível a partir do Java 8+, essa estrutura fecha o recurso automaticamente assim que o bloco é finalizado, mesmo que ocorra algum erro: [1]

java

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        try (Scanner scanner = new Scanner(System.in)) {
            System.out.println("Digite algo:");
            String texto = scanner.nextLine();
        } // O Scanner é fechado automaticamente aqui
    }
}
```

Use o código com cuidado.

⚠️ Observação importante sobre `System.in`

Se o seu `Scanner` estiver lendo do teclado (`new Scanner(System.in)`), **fechá-lo também fechará o fluxo de entrada padrão do sistema**, impedindo que você crie novos Scanners para ler do console no mesmo programa. Em pequenos exercícios de faculdade ou testes simples, você pode apenas ignorar esse aviso da IDE (como Eclipse ou IntelliJ) se for o caso. [1, 2]

Se quiser, me diga:

- Qual é o **contexto do seu código** (é um exercício simples ou uma aplicação maior)?
- Qual **IDE** você está utilizando?

Eu posso te ajudar a ajustar o trecho específico.

- Stack Overflow

  java - Vazamento de recursos: 'in' nunca é fechado

  Conteúdo traduzido — 1.439 1 1 distintivo de ouro 16 16 distintivos de prata 32 32 emblemas de bronze. Pergunta feita em 20 de setembro de 2012 às 19:1...

  

- GUJ.com.br

  Erro no scanner - GUJ

  O motivo desse warning é que o Scanner é um tipo de classe que precisa ser fechado após o uso para poder liberar os recursos que f...

  

- GUJ.com.br

  Problema na saida de dados Java - GUJ

  Aquele warning é porque você não chamou o método close do Scanner . Mas nem deve chamar, pois ele encapsula o System.in . Basta de...

  

porque é necessario fechar o scanner?

É necessário fechar o `Scanner` porque ele gerencia **recursos do sistema operacional** que estão fora do controle direto da máquina virtual do Java (JVM).

Quando você abre um `Scanner` para ler um arquivo, uma conexão de rede ou até mesmo o teclado (`System.in`), o sistema operacional reserva memória, ponteiros de arquivos e descritores de dados para essa operação. Se você não fechar o `Scanner`, esses recursos continuam presos.

Principais problemas de não fechar o Scanner

- **Vazamento de Memória (Memory Leak):** A memória associada ao fluxo de dados pode não ser liberada imediatamente pelo Garbage Collector, acumulando lixo na execução do programa.
- **Bloqueio de Arquivos:** Se o `Scanner` estiver lendo um arquivo e você não o fechar, o sistema operacional pode impedir que outros programas (ou o seu próprio) alterem ou deletem esse arquivo, pois ele ainda consta como "em uso".
- **Esgotamento de Recursos:** Sistemas operacionais têm um limite máximo de arquivos e conexões que podem ficar abertos simultaneamente. Se o seu código abrir vários Scanners (em um laço de repetição, por exemplo) e nunca fechá-los, o programa vai travar ao atingir esse limite.

Por que a IDE avisa mesmo usando `System.in` ?

O compilador e as IDEs (como Eclipse, NetBeans e IntelliJ) não sabem de onde o `Scanner` está puxando os dados. Elas apenas veem que uma classe que implementa a interface `AutoCloseable` foi aberta e não foi fechada. Por isso, elas exibem o alerta de forma preventiva para garantir que você siga as boas práticas.